

Projektmenedzsment. Költségbecslés, szoftvermérés

PROJEKTMENEDZSMENT

- szoftverfejlesztés sikerét nem csupán a technikai tudás, hanem a folyamat, a menedzsment és a technikai módszer egyensúlya határozza meg
 - a projekt sikertelenségét gyakran nem a programozók képzetlensége okozza, hanem a projektmenedzsment hibái
- a jó menedzselés nem garantálja a sikert, de a rossz menedzselés garantálja a bukást
- **projekt bukások okai:**
 - nem reális határidők
 - változó felhasználói / megrendelői követelmények
 - pénzügyi alulbecslések
 - kockázati tényezők
 - technikai nehézségek
 - rossz kommunikáció a csapatban
 - projekt menedzsment hibái
- **projektmenedzsment részei** (projektmenedzser feladatai)
 - emberek menedzselése
 - projekttervezése és ütemezése
 - kockázatmenedzsment: kockázatelemzés és folyamatos felülvizsgálat
 - költségbecslés
- a hatékony szoftverprojekt-menedzsment a 4P modellre épít

4P modell

- 4 alappillér: People (emberek), Product (termék), Process (folyamat), Project (projekt)
- 1. People (emberek, munkatársak)**
 - legfontosabb tényező
 - a human erőforrás a siker kulcs (SEI - Software Engineering Institute érettségi modelje is hangsúlyozza)
 - résztvevők (stakeholder-ek) kategóriái:
 - **felsővezetők:** üzleti döntéseket hozzák
 - **projektmenedzserek:** terveznek és irányítanak
 - **gyakorlati szakemberek:** fejlesztők, tervezők, tesztelők
 - **vevők / felhasználók:** akik megrendelik és használják a szoftvert
 - 2. Product (termék)**
 - 3. Process (folyamat)**
 - ki kell választani a megfelelő szoftverélelciklus-modellt (pl: agilis, vízesés) amely keretet ad a fejlesztésnek
 - 4. Project (projekt)**
 - a projekt maga az a tevékenységsorozat, amely a terméket előállítja

- Barry Boehm **W5HH** elve jó vázat ad a projekt definíciójához, érdemes minden projekt kezdetén megválaszolni
 - W5HH = Why, What, When, Who, Where, How, How much
 - Why = miért fejlesztjük? van-e üzleti indokoltsága?
 - What = mit fog csinálni a rendszer pontosan?
 - When = mikorra kell elkészülnie a terméknek vagy a mérföldköveknek?
 - Who = ki a felelős egyes funkciókért?
 - Where = hol helyezkednek el a felelősök (irodai vagy home office)?
 - How = hogyan zajlik a technikai és menedzsment munka? (milyen módszertannal)
 - How much = mennyi erőforrás (pénz, ember, idő) szükséges a megvalósításhoz?

Emberek menedzselése

- kommunikáció a megrendelővel és a csapattal
- **szellemi tőke:** az alkalmazottak tudása és tapasztalata, ezt a menedzsereknek jól kell kezelnie, mert a szoftverfejlesztő cég legnagyobb értéke ez (nem az iroda, se a hardver)
- sok projekt bukásának legfőbb oka a rossz humánmenedzsment
- **motivációs szintek:** a menedzsernek figyelnie kell a munkatársak szükségleteire ez az egyik legfontosabb dolog
 - szociális szükségletek (közösséghez tartozás)
 - megbecsülés (visszajelzés a jó munkáról)
 - önmegvalósítás (szakmai kihívások)
- fontos a **csapatszellem** kialakítása (“mi” tudat), ahol a csoport sikere fontosabb, mint az egyéni dicsőség, ez is a menedzser feladata
 - **csapatépítés**
 - közös tevékenységek (kirándulás, vacsora), melyek segítik a tagok közötti bizalom kialakulását
 - **munkakörnyezet**
 - fizikai környezet (nyitott vs zárt iroda) és CASE eszközök elérhetősége
- **emberek kiválasztása**
 - különböző tesztek: programozási képesség és pszichometrikus tesztek
 - sok tényező: alkalmazási terület, platform, programozási nyelv, kommunikációs készség, személyiség
 - a csapatot úgy kell felépíteni, hogy összeilleyenek és a projektnek is megfeleljenek
- a fejlesztői csapatszervezés struktúrája lehet:
 - **zárt forma** (centralizált (főprogramozói))
 - egy tapasztalt vezető hoz meg minden technikai döntést, a többiek az ő keze alá dolgoznak
 - hierarchikus
 - hatékony, de a vezető túlterhelődhet

- **véletlenszerű forma**
 - laza szerkezet, nincsenek szigorú szabályok
 - az egyéni kezdeményezések és az intuíció dominál
- **nyitott forma** (decentralizált (demokratikus))
 - nincs állandó vezető, a döntések konszenzussal születnek
 - előnye a magas morál
 - hátránya, hogy lassú a döntéshozatal
- **szinkronizált forma**
 - feladatok részekre bontják és az egyes részfeladatokat külön csoportok végzik
 - a csoportok között minimális a kommunikáció, csak az interfészek szintjén találkoznak

Projekttervezés

- egy iteratív folyamat, mely a projekt teljes élettartalma alatt tart, nem csak az elején
 - ahogy több információnk lesz, úgy a tervet folyamatosan pontosítani kell
- célja az erőforrások (emberek, pénz, eszközök) optimális elosztása a célok elérése érdekében
- főbb lépései
 - **megszorítások azonosítása** (határidő, költségvetés, létszám)
 - **paraméterek (méret) becslés**
 - mekkora lesz a szoftver?
 - mennyi ember kell hozzá?
 - **mérföldkövek és átadandók tervezése**
 - a menedzsernek látnia kell a haladást, ehhez mérhető pontokra van szükség
 - **mérföldkövek**
 - egy logikai egység vagy fázis vége
 - azt jelzi, hogy a projekt elért egy fontos szakaszhoz
 - nem mindig átadandó, lehet csak egy jelentés is
 - de ha mérföldkőhöz nem tartozik átadandó akkor nehéz igazolni, hogy a mérföldkő teljesült-e, ezért általában egy jól definiált mérföldkőhöz tartozik valamilyen dokumentum vagy kód
 - **átadandók**
 - a projekt során létrejövő kézzelfogható eredmény, amelyet általában a megrendelőnek adunk át
 - példa: tervdokumentáció, forráskód, telepíthető program
 - különbség: a mérföldkő egy esemény a naptárban, az átadandó pedig egy konkrét dolog, amit odaadunk a főnöknek vagy a vevőnek

- **projektütemezés, ütemterv készítés**

- feladatok sorrendbe állítása és felelősök kijelölése

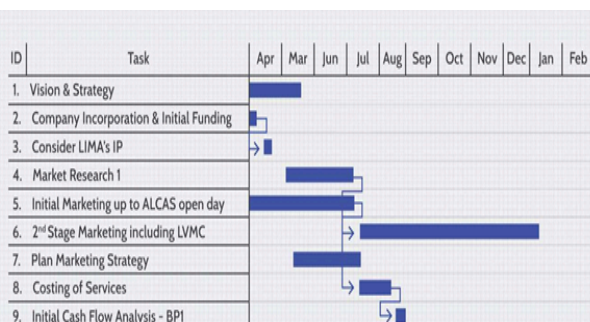
- meg kell becsülni a tevékenységekhez szükséges időt
 - mivel általában a kezdeti becslések optimisták ezért +50%-ot nyugodtan adhatunk rá
 - cél a munkaerő optimális kihasználása
 - a feladatok akár párhuzamosan is végezhetők
 - kritikus feladatokat meg kell keresni és előre venni őket

- **Ütemezési eszközök**

- **személy-hónap vagy ember-hónap**

- vannak változatai: nap, hét, hónap és év is lehet
 - emberek teljesítménye mérhető vele
 - azt adja meg, hogy hány ember hány havi munkája kell a feladat megoldásához
 - pl: 4 személy-hónap azt jelenti, hogy 1 ember dolgozik 4 hónapig, vagy 2 ember 2 hónapig
 - a fejlesztők száma és az idő nem fordítottan arányos (Brooks törvénye: "Késő projektbe embert tenni még tovább késlelteti a projektet" a kommunikációs többlet miatt)

- **Gantt-diagram (tevékenységdiagram)**



- tartalmazza a feladatok megvalósítására szánt/tervezett időt
 - függőleges tengelyen a feladatok
 - vízszintes tengelyen az idő tellik (napok, hetek, hónapok)
 - a vízszintes sávok hossza a feladat várható időtartamát jelenti
 - a nyilak jelzik a feladatok közötti függőségeket (pl: tesztelés nem kezdődhet el, amíg a kódolás nincs kész)

- **felülvizsgálat (kockázatelemzés), újratervezés**

- a bölcs projektvezető pesszimista
 - kockázatelemzés: mi történhet, ami hátráltatja a projektet?

- **projektterv megírása (dokumentum előállítás)**

Kockázatmenedzsment

- a kockázat egy jövőbeli bizonytalan esemény, mely ha bekövetkezik akkor negatív hatással lehet a projektre
 - pl: kulcsembert távozása, technológiai hiba
- két stratégiát különböztetünk meg:
 - **reaktív** (tűzoltás)
 - akkor cselekszünk, amikor a baj már megtörtént
 - **proaktív**
 - előre azonosítjuk a kockázatot, megbecsüljük a bekövetkezés valószínűségét és hatását, majd tervet dolgozunk ki az elkerülésére

Szoftverkonfiguráció-kezelés (SCM)

- a szoftver folyamatosan változik fejlesztés közben, a változások kontrollálása kritikus
- SCM elemei
 - **verziókövetés**
 - a forráskód történetének tárolása (git)
 - lehetővé teszi a visszatérést korábbi verziókhoz, állapotokhoz
 - **változáskezelés**
 - hivatalos folyamat a módosítási kérelmek (Change Request) elbírálására
 - lépései: Elemzés → Hatásvizsgálat → Költségbecslés → Jóváhagyás → Implementálás

SZOFTVERMÉRÉS ÉS METRIKÁK

- a fejlesztés menedzseléséhez mérnünk kell
- **Mérték**
 - nyers adat
 - pl: hibák száma
- **Metrika**
 - viszonyított mérőszám, amely a rendszer vagy folyamat tulajdonságait méri
 - pl: hibák száma / 1000 sor kód (hibasűrűség)
 - **Vezérlési metrikák**
 - folyamattal kapcsolatosak
 - pl: egy hiba javításához szükséges átlagos idő (folyamat és projekt metrikák)
 - régi projektek mérési adatait használjuk új projektek költség- és időbecslésére
 - **folyamat metrikák**
 - az aktuális folyamat hatékonyságát méri
 - menedzsment látja, hogy mi az, ami jól megy és mi nem
 - **projekt metrikák**
 - a jelenlegi projekt státusza
 - kockázatok és kritikus problémák feltárása
 - projekt csapat képességeinek értékelése
 - szoftver minőség (hibák száma)
 - termelékenység (sorokszáma, dokumentáció soronként / személyhónap)
 - **Prediktor metrikák**
 - termékkel kapcsolatosak (termék metrikák)
 - pl: LOC, ciklomatikus komplexitás, osztály metódusainak száma
 - mindkét felsorolt típus befolyásolja a vezetői döntéshozatalt:
Leletkészítés → Diagnózis → Kezelés (vagy Megelőzés)

Mérési kategóriák

- **Direkt mérés**
 - közvetlenül megfigyelhető tulajdonságok
 - pl: kódsorok száma (LOC - Lines of Code), végrehajtási sebesség, hibák száma
- **Indirekt mérés**
 - közvetve következtetett minőségi jellemzők
 - pl: funkcionalitás, bonyolultság, megbízhatóság, karbantarthatóság

Méret alapú metrikák (LOC - Lines of Code)

- több technika:
 - csak a nem üres sorok
 - csak a végrehajtható sorok
 - dokumentáció mérete
- legegyszerűbb, de egyben legvitatottabb metrika is
 - előnye: könnyen számolható
 - hátránya: nyelvfüggő (ugyanaz a funkció assembly-ben 100 sor, míg Python-ban 5 sor), ezért félrevezető lehet
- LOC-ból származtatott metrikák
 - Hibasűrűség (Hiba / KLOC)
 - Termelékenység (LOC / személyhónap)
 - hány sor kód egy hónap alatt

Funkció pont alapú metrikák (Function Point - FP)

- nyelvfüggetlen módszer
- a szoftver funkcionalitását méri a felhasználó szemszögéből
- öt összetevőt mérünk (mennyi van):
 - felhasználói inputok (bemenetek) száma pl: adatfelviteli képernyők
 - felhasználói outputok (kimenetek) száma pl riportok, képernyők, hibaüzenetek
 - felhasználói lekérdezések száma (interaktív válaszok)
 - fájlok száma (adatok logikai csoportja, a rendszer által kezelt adatbázis táblák)
 - külső interfészek száma (adatfájlok, ami adatokat továbbít)
- menete
 - megszámloljuk az elemeket és súlyozzuk őket bonyolultságuk szerint
 - egyszerű, átlagos, komplex
 - ebből megkapjuk az **UFP-t** (Unadjusted Function Points)
 - vannak további módosító tényezők, de ezek nagyon szubjektívek
 - a projekt összetettsége és a teljesítmény
- előnyei
 - már a tervezési fázisban becsülhető, pl: a LOC nem, mert kezdetben nem tudjuk hány sor lesz a kód
 - összehasonlíthatóvá tesz különböző nyelveken írt projekteket is

Bonyolultsági metrikák: McCabe-féle ciklomatikus szám

- a szoftver logikai komplexitását méri a vezérlési gráf alapján: $V(G) = E - N + 2$
 - ahol E az élek száma, N a csomópontok száma
 - a gráfban található függetlenkörök számát adja meg, azaz a kódban a döntési pontok számát
- ha $V(G) > 10$, akkor a modul túl bonyolult, nehezen tesztelhető ezért javasolt a felbontása

Minőségi metrikák

- a szoftver minőségét nem lehet egyetlen számmal leírni ezért több mutatót van:
 - **helyesség**
 - a szoftver mennyire felel meg a specifikációnak
 - mérőszáma a hibasűrűség: hiba / KLOC
 - **karbantarthatóság**
 - milyen könnyű módosítani a szoftveren
 - mérőszáma: MTTC (Mean-Time-To-Change), azaz a változáshoz szükséges átlagos idő
 - **integritás**
 - ellenállóképesség a támadásokkal és illetéktelen hozzáféréssel szemben
 - **használhatóság**
 - felhasználói felület ergonómiája
 - mérőszáma: mennyi idő alatt tanulja meg egy új felhasználó a rendszer használatát

DRE (Defect Removal Efficiency) mutató

- egy minőségi metrikához tartozó mutató, azt jelzi, hogy a fejlesztési folyamat során milyen hatékonysággal szűrjük ki a hibákat az átadás előtt
- $DRE = E / (E + D)$
 - **E**: fejlesztés közben (átadás előtt) megtalált hibák száma
 - **D**: a felhasználó által (átadás után) megtalált hibák száma
 - ideális az ha minnél közelebb van az 1-hez (100%) annál jobb a minőségbiztosítás
- pl: ha a tesztelők találtak 90 hibát, de a felhasználó még jelzett 10-et a kiadás után, akkor a $DRE = 90 / (90 + 10) = 0.9$, azaz 90%-os a szűrési hatékonyságunk

Folyamatfejlesztés és minőség

- **CMMI (Capability Maturity Model Integration)**

- segít a cégeknek javítani a fejlesztési folyamataikat
- 5 szintet különböztetünk meg

- 1. kezdeti**

- ad-hoc folyamatok, káosz
- a siker az egyéneken múlik (“hősök”, jó teljesítménnyel)

- 2. reprodukálható**

- alapvető projektmenedzsment már van (ütemezés, költségkövetés)

- 3. definiált**

- a cég egységes, dokumentált folyamatokkal rendelkezik

- 4. ellenőrzött**

- a folyamatokat mérjük és statisztikai módszerekkel kontrolláljuk

- 5. optimalizált**

- folyamatos önfejlesztés, mérések alapján javítjuk a módszereket

KÖLTSÉGBECSLÉS

- a projekt egyik legnehezebb része az erőforrásigény (**idő + munka + pénz**) előrejelzése
- a becslés pontossága a projekt előrehaladtával nő
 - **becslési (bizonytalansági) tölcsérnek** nevezzük ezt a jelenséget, azaz kezdetben nehéz a becslés mert nagy a bizonytalanság
- **Költségtényezők**
 - **hardver és szoftver költségek**
 - a szerverek, a munkaállomások és különböző szoftver licencek (pl: IDE, adatbáziskezelő, stb)
 - **utazási és képzési költségek**
 - fejlesztők betanítása az új technológiákra vagy utazás a megrendelőhöz
 - **munkaköltség**
 - legjelentősebb tétel (általában 80%-a a költségeknek)
 - fejlesztők, tesztelők, menedzserek és egyéb kisegítő személyzet fizetése, járulékok, adók, iroda bérleti díja és rezsi (fűtés, villany)
- **Becslési technikák**
 - **szakértői becslés (vélemény)**
 - tapasztalt kollégák intuíciója alapján
 - pl: Delphi módszer
 - **dekompozíció**
 - a problémát kisebb részekre bontjuk, ezeket külön becsüljük, majd összegezzük
 - **probléma alapú**
 - szoftvert funkciókra bontjuk
 - pl: LOC vagy Function Point alapján tudunk becsülni
 - **folyamat alapú**
 - a fejlesztési folyamat lépéseire (tervezés, kódolás, tesztelés) becslünk ráfordítást
 - **analógia alapú becslés (empirikus modellek)**
 - múltbeli adatokból (korábbi hasonló projekt adatait vesszük alapul) statisztikai képleteket alkotunk
 - Parkinson törvénye
 - itt a becslés nem a feladaton, hanem a rendelkezésre álló erőforráson alapul
 - (pl. ha van 3 hónap és 5 ember, akkor ennyit mondunk)
 - veszélyes, mert nem a valós igényt tükrözi
 - „Pricing to win” (Árazás a győzelemhez)
 - olyan árat mondunk, amit a vevő hajlandó kifizetni
 - kockázatos, mert a fejlesztés többbe kerülhet, mint a bevétel

COCOMO modell (Constructive Cost Model)

- legelterjedtebb algoritmusos modell
- költségek és időtartamok becslésére jó szoftverfejlesztésben
 - a sima COCOMO model a kódsorok száma alapján ad egy becslést
- $E = a_b \cdot (KLOC)^{b_b}$
 - alapja a kódsorok száma (KLOC = 1000 Lines of Code), a ráfordítást (E) emberhónapban méri
 - ahol az a_b és b_b konstansok a projekt típusától függenek
- **Projekt típusok**
 - **organic (szerves)**
 - kicsi, egyszerű projekt (2 - 50 KLOC)
 - tapasztalt csapat
 - stabil követelmények
 - legolcsóbb típus
 - **semi-detached (félig független)**
 - közepes méret és bonyolultság (50 - 300KLOC)
 - **embedded (beágyazott)**
 - szigorú hardveres/szoftveres megköötések, komplex rendszer (pl: repülésirányítás) (300 fölötti KLOC)
 - ez igényli a legtöbb ráfordítást
- az egész szoftvert modulokra bontjuk és úgy számoljuk ki a ráfordítást (E) külön külön modulokra, majd összegezzük ezeket
- **Lépései**
 - tervezés és megszorítások (requirements)
 - rendszertervezés
 - specifikáció minden komponensre
 - modulok kódolása és tesztelése
 - integráció és tesztelés
 - COCOMO modellel kiszámítjuk a ráfordítást
- **COCOMO II**
 - COCOMO továbbfejlesztése, mely figyelembe veszi a modern fejlesztési eszközöket és az újrafelhasználást is
 - a becslést nem csupán a kódsorok száma (KLOC) alapján adja, hanem tovább pontosítják objektumpontok (költségtényezők segítségével)
 - pl: csapat képességei, szoftver megbízhatósági igényei vagy a fejlesztői környezet minősége, fejlesztési módszertan, stb
 - iparban ez a használt
$$PM = (obj.pont) * [(100 - \%újrahaszn)/100] / PROD$$
 - PM: munkamennyiség (emberhónap)
 - PROD: termelékenység (obj.pont/hónap)

- **3 alverziója:**

- **alapmodell**

- projekt méretére, a termelékenységre és alapvető tényezőkre koncentrálnak

- **köztes modell**

- részletesebb becslést ad mint az alap
 - figyelembe veszi még a projektek specifikus tulajdonságait és körülményeit is

- **részletes modell**

- legpontosabb, komplex verzió
 - még részletesebb projekt tulajdonságokat is figyelembe vesz a becsléshez

Make / Buy döntés

- gyakori dilemma, hogy fejlesszük-e a szoftvert (Make) vagy vegyünk egy kész megoldást (Buy)
- a döntéshez nemcsak a beszerzési árat, hanem a tesztelés, karbantartás és a bevezetés költségét is össze kell hasonlítani a belső fejlesztés költségével